



universidad
de león



Escuela de Ingenierías Industrial, Informática y Aeroespacial

GRADO EN INGENIERÍA DE DATOS E INTELIGENCIA ARTIFICIAL

Trabajo de Fin de Grado

Sistemas de Movilidad Autónoma en Simuladores Urbanos mediante Redes Neuronales y Algoritmos Evolutivos

Autonomous Mobility Systems in Urban Simulators Using Neural Networks and
Evolutionary Algorithms

Autor: Sergio Alves Martín
Tutor@s: Sergio Rubio Martín / Alicia Merayo Corcoba

(Junio, 2026)

UNIVERSIDAD DE LEÓN
Escuela de Ingenierías Industrial, Informática y
Aeroespacial

GRADO EN INGENIERÍA DE DATOS E
INTELIGENCIA ARTIFICIAL
Trabajo de Fin de Grado

ALUMNO: Sergio Alves Martín

TUTOR@S: Sergio Rubio Martín / Alicia Merayo Corcoba

TÍTULO: Sistemas de Movilidad Autónoma en Simuladores Urbanos mediante Redes Neuronales y Algoritmos Evolutivos

TITLE: Autonomous Mobility Systems in Urban Simulators Using Neural Networks and Evolutionary Algorithms

CONVOCATORIA: Junio, 2026

RESUMEN:

El trabajo desarrolla un sistema de movilidad autónoma para *CognitiveCarSimulatorReloaded*, simulador urbano en Unreal Engine 5 orientado a rehabilitación y valoración cognitiva con ASPAYM. La solución incorpora vehículos con sensores simulados, interpretación de señales y un controlador neuronal entrenado con algoritmos evolutivos, junto con persistencia de modelos y análisis de métricas CSV. La evaluación de referencia incluye 2067 generaciones de entrenamiento y seis sesiones de test con 1560 evaluaciones: fitness máximo de 112447.60 en entrenamiento, 85530.55 en test y una tasa agregada de acierto del 25.19 %.

ABSTRACT:

This thesis develops an autonomous mobility system for *CognitiveCarSimulatorReloaded*, an Unreal Engine 5 urban simulator aimed at cognitive rehabilitation and assessment with ASPAYM. The solution adds vehicles with simulated sensors, traffic-sign interpretation and a neural controller trained through evolutionary algorithms, plus model persistence and CSV metric analysis. The reference evaluation comprises 2067 training generations and six test sessions with 1560 evaluations: maximum fitness values of 112447.60 in training and 85530.55 in testing, with an aggregate success rate of 25.19 %.

Palabras clave: simulación urbana, movilidad autónoma, redes neuronales, algoritmos evolutivos, Unreal Engine.

Firma del alumno:

VºBº Tutor@s:

Resumen

CognitiveCarSimulatorReloaded es un simulador urbano desarrollado en Unreal Engine 5 para apoyar actividades de rehabilitación y valoración cognitiva relacionadas con la conducción. Este Trabajo de Fin de Grado aborda una de las carencias detectadas en la versión previa del sistema: la ausencia de tráfico autónomo capaz de generar situaciones dinámicas y repetibles. El objetivo es diseñar e integrar una arquitectura de movilidad vehicular que perciba el entorno, actúe sobre un vehículo con físicas reales, aprenda mediante experimentación y produzca información suficiente para evaluar su comportamiento. La solución combina once sensores distribuidos en un campo de visión de 180 grados, variables dinámicas del vehículo y contexto de semáforos, stops y cedas. Estas entradas alimentan un perceptrón multicapa ligero cuya configuración actual contiene 22 entradas, una capa oculta de 16 neuronas y dos salidas continuas para dirección y aceleración o frenado. La política de control se optimiza mediante un algoritmo evolutivo que genera poblaciones, conserva modelos, muta pesos y selecciona individuos según una función de aptitud instrumentada. El sistema incluye además persistencia de modelos, separación entre puntos de entrenamiento y test, selección de trayectorias en intersecciones y actores específicos para validar el cumplimiento de señales. Para facilitar la trazabilidad, cada ejecución exporta métricas detalladas a CSV y se analiza mediante scripts de Python que comprueban la calidad de los datos y generan informes y visualizaciones. La evaluación de referencia utiliza una sesión de entrenamiento de 2067 generaciones y 66144 registros, junto con seis sesiones de test que suman 1560 evaluaciones sobre 52 puntos de aparición. Se alcanzan valores máximos de fitness de 112447.60 en entrenamiento y 85530.55 en test. La tasa agregada de acierto es del 25.19 %, con un intervalo de confianza del 95 % entre el 23.10 % y el 27.41 %. Frente a los bloques de test de los días 2 y 5 de junio, aumentan la tasa de acierto, el fitness medio y el tiempo de vida. El entrenamiento elimina el estancamiento LAZY y reduce los fallos legales, aunque no presenta convergencia clara y mantiene posibles atascos ante señales. Las colisiones en intersecciones y la variabilidad entre puntos de aparición continúan siendo las principales limitaciones. Por ello, el trabajo se presenta como una base funcional y medible sobre la que seguir mejorando percepción, entrenamiento dirigido, interacción entre vehículos y validación en escenarios de uso más amplios.

Abstract

CognitiveCarSimulatorReloaded is an urban driving simulator developed in Unreal Engine 5 to support cognitive rehabilitation and assessment activities related to driving. This Bachelor's Thesis addresses one of the main limitations identified in the previous version of the system: the lack of autonomous traffic capable of producing dynamic and repeatable situations. The objective is to design and integrate a vehicle mobility architecture that perceives its surroundings, controls a physically simulated car, learns through experimentation and produces enough information to evaluate its behaviour. The proposed solution combines eleven sensors distributed across a 180-degree field of view, dynamic vehicle variables and contextual information from traffic lights, stop signs and yield signs. These inputs feed a lightweight multilayer perceptron whose current configuration contains 22 inputs, one hidden layer with 16 neurons and two continuous outputs for steering and acceleration or braking. The control policy is optimised through an evolutionary algorithm that creates populations, preserves models, mutates weights and selects individuals according to an instrumented fitness function. The system also provides model persistence, separate training and test spawn points, route selection at intersections and dedicated actors for validating compliance with traffic controls. To support traceability, every run exports detailed metrics to CSV files, which are processed by Python scripts that validate data quality and generate reports and visualisations. The reference evaluation comprises a training session with 2067 generations and 66144 vehicle records, together with six test sessions totalling 1560 evaluations across 52 spawn points. These experiments achieve maximum fitness values of 112447.60 during training and 85530.55 during testing. The aggregate success rate is 25.19 %, with a 95 % confidence interval from 23.10 % to 27.41 %. Compared with the test blocks run on 2 and 5 June, success rate, mean fitness and survival time all increase. Training removes LAZY stagnation and reduces legal failures, although it does not show clear convergence and still presents possible traffic-control traps. Collisions at intersections and performance variability between spawn points remain the main limitations. The work should therefore be understood as a functional and measurable baseline for further improvements in perception, targeted training, interaction between vehicles and validation under broader usage scenarios.

Índice general

1. Estudio del problema	1
1.1. Contexto	1
1.2. Problema que se resuelve	2
1.3. Objetivo general	2
1.4. Objetivos específicos	3
1.5. Estructura del trabajo	3
2. Estado del arte	4
2.1. Simuladores de conducción	4
2.2. Agentes autónomos en entornos urbanos	4
2.3. Redes neuronales y neuroevolución	5
2.4. Retos específicos	6
3. Tecnologías y herramientas	7
3.1. Unreal Engine 5	7
3.2. Sistema de físicas Chaos Vehicle	7
3.3. Raycasting y percepción local	8
3.4. Red neuronal	8
3.5. Algoritmos evolutivos	9
3.6. Análisis de datos	9
3.7. Control de versiones	10
4. Gestión del proyecto	11
4.1. Metodología	11
4.2. Cronología de decisiones	11
4.3. Alcance	12
4.4. Gestión de riesgos	13
4.5. Estimación de recursos y presupuesto	14
5. Núcleo del trabajo	15
5.1. Descripción general de la solución	15
5.2. Subsistemas	15
5.2.1. Gestor evolutivo	15
5.2.2. Controlador de IA y red neuronal	16
5.2.3. Percepción mediante sensores	17
5.2.4. Sistema de fitness	18
5.2.5. Semáforos	18

5.2.6. Stop y ceda el paso	19
5.3. Pipeline experimental	19
6. Resultados	20
6.1. Datos analizados	20
6.2. Resumen cuantitativo	20
6.3. Evolución del entrenamiento	20
6.4. Evaluación de test	22
6.5. Razones de finalización	23
6.6. Comportamiento ante señalización	23
6.7. Dificultad por puntos de aparición	23
6.8. Conclusión experimental	24
7. Normativa y accesibilidad	25
7.1. Protección de datos	25
7.2. Seguridad y uso responsable	25
7.3. Accesibilidad	26
8. Conclusiones y líneas futuras	27
8.1. Conclusiones	27
8.2. Aportaciones realizadas	28
8.3. Problemas encontrados	28
8.4. Líneas futuras	29
Agradecimientos	31
Bibliografía	32
A. Fuentes analizadas	A-1
A.1. Documentación administrativa y de formato	A-1
A.2. Actas de reunión	A-1
A.3. Descripciones del programa	A-2
A.4. Resultados experimentales	A-2

Índice de figuras

Figura 6.1. Tendencia del fitness en el entrenamiento del 8 de junio de 2026. . 21

Índice de tablas

Tabla 4.1.	Principales hitos y decisiones documentadas en reuniones.	11
Tabla 4.2.	Riesgos principales del proyecto.	13
Tabla 6.1.	Resultados principales del entrenamiento y del bloque de test del 8 de junio de 2026.	21
Tabla 6.2.	Evolución de los bloques de test disponibles en junio de 2026. . . .	22

Glosario

ASPAYM	Asociación/Fundación colaboradora orientada a mejorar la calidad de vida de personas con lesión medular, gran discapacidad física y daño cerebral adquirido.
Blueprint	Sistema visual de programación de Unreal Engine empleado para implementar lógica de juego, interacción, controladores y sistemas de simulación.
Chaos Vehicle	Sistema de físicas de Unreal Engine 5 utilizado para simular el comportamiento dinámico del vehículo.
DCA	Daño Cerebral Adquirido.
Fitness	Función de aptitud que cuantifica el comportamiento de cada vehículo autónomo durante una generación evolutiva.
NavMesh	Malla de navegación utilizada por motores de videojuegos para calcular rutas transitables. En este proyecto fue estudiada y posteriormente descartada como solución principal para vehículos.
NPC	<i>Non-Player Character</i> ; agente controlado por el sistema y no por el usuario.
Raycasting	Técnica de consulta geométrica que lanza rayos o trazas desde un actor para detectar obstáculos y distancias en el entorno.
RN	Red neuronal artificial.
UE5	Unreal Engine 5.

Capítulo 1

Estudio del problema

1.1 CONTEXTO

La conducción es una actividad compleja que exige coordinación motora, atención sostenida, atención dividida, interpretación de señales, percepción espacial y toma de decisiones bajo restricciones temporales. En personas que han sufrido daño cerebral adquirido, estas capacidades pueden verse alteradas aunque existan adaptaciones físicas que permitan manejar un vehículo. Por ello, antes de exponer al paciente a situaciones reales de tráfico, resulta de interés disponer de entornos controlados donde observar, entrenar y valorar comportamientos de conducción de forma segura.

El proyecto *CognitiveCarSimulatorReloaded* nace dentro de esta necesidad. La propuesta original define un simulador realista de conducción urbana para entrenamiento y valoración de alteraciones cognitivas tras un daño cerebral adquirido, desarrollado en colaboración con ASPAYM [1]. La versión inicial del simulador, desarrollada durante el curso 2024-2025, ya incluía un entorno urbano, vehículo, interacción física, señalización y registro de datos [2]. Sin embargo, las evaluaciones posteriores identificaron una carencia clave: el escenario necesitaba tráfico autónomo y dinámico para trasladar al usuario a una situación más realista.

El presente TFG aborda esa carencia desde el ámbito de la Ingeniería de Datos e Inteligencia Artificial. El foco no se sitúa en crear un simulador desde cero, sino en diseñar e integrar modelos de movilidad autónoma capaces de aprender comportamientos de conducción dentro de un entorno urbano ya existente. El trabajo combina redes neuronales, algoritmos evolutivos, sensores simulados, control físico en tiempo real y análisis de datos experimentales.

1.2 PROBLEMA QUE SE RESUELVE

El problema técnico principal consiste en conseguir que agentes autónomos, inicialmente vehículos, circulen por un mapa urbano de forma suficientemente estable, medible y compatible con las normas básicas de tráfico. A diferencia de un recorrido prefijado, un agente autónomo debe responder a múltiples condiciones locales: límites de la calzada, obstáculos, intersecciones, semáforos, señales de stop, señales de ceda el paso y cambios en su propio estado dinámico.

Durante el desarrollo se evaluaron soluciones deterministas de navegación, como el uso de NavMesh o recorridos mediante *Path Finding*. Estas aproximaciones resultaron útiles como punto de partida, pero presentaron limitaciones para el objetivo clínico y experimental del simulador: rutas rígidas, giros poco naturales, dificultad para representar la toma de decisiones local y menor capacidad de adaptación a situaciones imprevistas. Como consecuencia, el proyecto evolucionó hacia una solución basada en percepción local mediante sensores y aprendizaje evolutivo.

Además, el problema requiere que el comportamiento obtenido sea observable y explicable. No basta con que los vehículos se desplacen por el escenario; es necesario registrar por qué avanzan, se detienen, colisionan, incumplen una señal o completan correctamente una maniobra. Esta trazabilidad condiciona la arquitectura de la solución, porque cada decisión de control debe poder relacionarse con sensores, contexto de tráfico, componentes de fitness y resultados experimentales.

1.3 OBJETIVO GENERAL

El objetivo general del trabajo es desarrollar e integrar un sistema de movilidad autónoma para un simulador urbano de conducción, basado en redes neuronales y algoritmos evolutivos, que permita entrenar, evaluar y analizar vehículos autónomos dentro de Unreal Engine 5.

De forma complementaria, el proyecto busca dejar una base reutilizable para futuras ampliaciones del simulador. La movilidad vehicular autónoma actúa como primer bloque técnico sobre el que podrán incorporarse perfiles de conducción, otros agentes dinámicos y escenarios de validación más cercanos al uso clínico previsto.

1.4 OBJETIVOS ESPECÍFICOS

El objetivo general se concreta en tareas que cubren el ciclo completo de desarrollo: conocer el punto de partida, construir el controlador, entrenarlo y comprobar su comportamiento con datos. No se plantea únicamente obtener vehículos que se desplacen por el mapa; también se busca que las decisiones de diseño queden justificadas y que los resultados puedan repetirse y compararse. A partir de este criterio, los objetivos específicos son:

- Analizar el estado inicial del simulador y las restricciones técnicas heredadas de la versión previa.
- Diseñar una arquitectura de percepción para vehículos autónomos basada en sensores simulados mediante raycasting.
- Implementar una red neuronal ligera capaz de transformar entradas sensoriales y de contexto en acciones de conducción.
- Diseñar un proceso de entrenamiento evolutivo que evalúe poblaciones de vehículos, seleccione individuos destacados y aplique mutaciones sobre sus pesos.
- Integrar señales de tráfico, semáforos, stops y cedas en la lógica de percepción, parada y validación de la IA.
- Construir una función de fitness interpretable, con componentes positivos y penalizaciones, que permita medir progreso, errores y comportamientos no deseados.
- Generar registros experimentales en CSV y analizarlos mediante scripts reproducibles para evaluar entrenamiento y test.
- Identificar limitaciones, riesgos y líneas futuras para ampliar la movilidad autónoma a perfiles de conductor, peatones, bicicletas y otros agentes.

1.5 ESTRUCTURA DEL TRABAJO

El documento se organiza en ocho capítulos principales. Tras este estudio del problema, el capítulo 2 resume el estado del arte y los retos asociados a simuladores, agentes autónomos y aprendizaje evolutivo. El capítulo 3 describe las tecnologías y herramientas utilizadas. El capítulo 4 presenta la gestión del proyecto, la evolución temporal, los riesgos y una estimación preliminar de recursos. El capítulo 5 desarrolla el núcleo técnico de la solución. El capítulo 6 recoge los resultados experimentales. El capítulo 7 trata normativa, protección de datos y accesibilidad. Finalmente, el capítulo 8 expone conclusiones, aportaciones y líneas futuras.

Capítulo 2

Estado del arte

2.1 SIMULADORES DE CONDUCCIÓN

Los simuladores de conducción permiten reproducir situaciones de tráfico en un entorno controlado, repetible y seguro. En el ámbito formativo se utilizan para entrenar maniobras, respuesta ante eventos inesperados y familiarización con normas de circulación. En el ámbito sanitario, su valor reside en que permiten observar capacidades cognitivas y motoras sin exponer al usuario a riesgos reales. La versión previa de *CognitiveCarSimulator* ya se apoyaba en esta idea: un entorno urbano en Unreal Engine donde registrar tiempos de reacción, velocidades, colisiones y datos exportables para especialistas [2].

Para que un simulador de rehabilitación resulte convincente, el entorno no puede limitarse a una vía vacía. El usuario debe compartir la escena con otros agentes que creen carga cognitiva: vehículos que se detienen, peatones que cruzan, ciclistas, señales y elementos dinámicos. La propuesta de *CognitiveCarSimulatorReloaded* identifica precisamente esa necesidad al plantear IAs conductoras configurables, peatones, bicicletas y animales como ampliaciones progresivas del simulador [1].

2.2 AGENTES AUTÓNOMOS EN ENTORNOS URBANOS

La navegación autónoma en videojuegos y simulación suele abordarse mediante técnicas deterministas, como grafos de rutas, splines o mallas de navegación. Estas técnicas son robustas cuando el objetivo es desplazar agentes por zonas transitables con trayectorias controladas, pero pueden producir comportamientos rígidos si se aplican directamente a vehículos físicos. Un coche no se desplaza como un personaje humano: tiene inercia, radio de giro, aceleración, frenada, fricción y restricciones de control continuas.

En este proyecto se comprobó esa diferencia durante las primeras iteraciones. El uso de NavMesh y Path Finding fue estudiado para permitir recorridos por la ciudad, pero las reuniones de seguimiento y los informes técnicos documentan problemas de rigidez, dificultad en intersecciones y falta de naturalidad. Por ello se adoptó una estrategia más cercana a la percepción local: el agente no sigue una ruta global cerrada, sino que interpreta distancias, estado del tráfico y contexto normativo para producir acciones continuas.

2.3 REDES NEURONALES Y NEUROEVOLUCIÓN

Las redes neuronales artificiales permiten aproximar funciones complejas a partir de entradas numéricas. En este TFG se emplean como política de control: reciben sensores, velocidad, contexto de señalización y estado de parada, y producen dirección y aceleración/frenado. La configuración vigente es deliberadamente ligera para poder ejecutarse en tiempo real sobre múltiples vehículos: 22 entradas, una capa oculta de 16 neuronas y dos salidas continuas con activación hiperbólica.

El entrenamiento no se plantea como aprendizaje supervisado, ya que no se dispone de un conjunto amplio de trayectorias humanas etiquetadas dentro del simulador. En su lugar, se adopta un algoritmo evolutivo. Cada individuo de una población representa un conjunto de pesos de la red neuronal. La simulación evalúa el comportamiento mediante fitness; los mejores pesos se conservan, se mutan y se reutilizan en generaciones posteriores. Este enfoque resulta adecuado para explorar comportamientos en entornos donde la señal de aprendizaje procede de la interacción con el mundo simulado.

La elección de un enfoque evolutivo también facilita trabajar con objetivos parcialmente contradictorios. Un vehículo debe avanzar, pero no a cualquier precio; debe mantener velocidad útil, pero frenar ante obstáculos; debe explorar la ciudad, pero respetar stops, cedas y semáforos. En este contexto, el fitness actúa como mecanismo de equilibrio entre progreso, seguridad, cumplimiento normativo y estabilidad de la conducción.

2.4 RETOS ESPECÍFICOS

La revisión de las alternativas disponibles y las primeras pruebas sobre el simulador permitieron acotar los problemas que condicionaban la solución. Algunos proceden del propio vehículo físico y otros aparecen al trasladar un método de aprendizaje a un mapa urbano con cruces y señalización. Los principales retos identificados son:

- **Simulación física:** el control neuronal debe traducirse a entradas de un vehículo con físicas reales, evitando aceleraciones, frenadas o giros no plausibles.
- **Intersecciones:** los cruces concentran la mayor complejidad, porque combinan decisión de trayectoria, detección de límites, prioridades y otros vehículos.
- **Diseño de fitness:** una función de aptitud mal calibrada puede premiar estrategias no deseadas, como detenerse indefinidamente, avanzar demasiado despacio o compensar errores graves con comportamientos parciales.
- **Generalización por puntos de aparición:** el modelo debe funcionar en diferentes zonas del mapa, no únicamente en los puntos de entrenamiento más sencillos.
- **Coste computacional:** entrenar decenas de vehículos simultáneamente en Unreal Engine implica carga de CPU, GPU y física.
- **Trazabilidad:** al tratarse de un sistema experimental, es imprescindible registrar métricas suficientes para explicar por qué una iteración mejora o empeora.

Estos retos aparecen de forma combinada durante el desarrollo. Una mejora en sensores puede modificar el fitness, un cambio en intersecciones puede alterar la tasa de colisiones y una reducción de coste computacional puede afectar al número de generaciones evaluables. Por ello, el proyecto no se aborda como una única implementación cerrada, sino como un proceso de ajuste continuo apoyado en métricas y revisión experimental.

Capítulo 3

Tecnologías y herramientas

3.1 UNREAL ENGINE 5

Unreal Engine 5 es el motor sobre el que se desarrolla el simulador. Proporciona renderizado 3D, editor visual, sistema de actores, colisiones, físicas, Blueprints y herramientas de depuración [3]. Su elección viene heredada de la versión inicial del proyecto y resulta coherente con los requisitos de realismo visual, interacción en tiempo real y despliegue del simulador como aplicación ejecutable.

El trabajo se implementa principalmente mediante Blueprints, lo que permite integrar lógica de IA, controladores, sensores y eventos de tráfico dentro del propio editor. Este enfoque facilita la iteración rápida, aunque introduce retos de mantenibilidad cuando los grafos crecen. Por ello se han documentado los Blueprints principales del gestor evolutivo, el controlador de IA, el vehículo autónomo y los actores de señalización. Las denominaciones concretas de estos módulos se recogen en el anexo de fuentes para mantener la trazabilidad técnica sin sobrecargar el texto principal.

3.2 SISTEMA DE FÍSICAS CHAOS VEHICLE

El vehículo se controla mediante el sistema de físicas Chaos Vehicles de Unreal Engine [4]. La red neuronal no mueve directamente el actor, sino que genera consignas continuas que se aplican al componente de movimiento del vehículo. La salida de dirección se envía como *steering input*; la salida de aceleración/frenado se interpreta de forma separada: valores positivos se aplican como acelerador y valores negativos como freno.

Esta separación es importante porque mantiene el comportamiento dentro de la física del motor. El agente aprende a actuar sobre el vehículo, pero la respuesta final depende de velocidad, fricción, inercia, contacto con el suelo y restricciones propias del modelo físico.

3.3 RAYCASTING Y PERCEPCIÓN LOCAL

La percepción del vehículo se implementa mediante trazas o rayos simulados. El controlador lanza once sensores en un ángulo de visión de 180 grados desde el vehículo. Cada sensor devuelve una distancia normalizada al primer obstáculo relevante, con valor cercano a 1 cuando no se detecta un obstáculo próximo y valores menores cuando existe un límite, pared, vehículo o borde de intersección.

Además de los sensores laterales y frontales, se calculan variables derivadas como la distancia frontal a obstáculo, la distancia del sensor izquierdo, la distancia del sensor derecho y la diferencia horizontal entre ambos. Estas magnitudes permiten construir una señal de centrado en carril y detectar correcciones equivocadas.

3.4 RED NEURONAL

La red neuronal empleada es un perceptrón multicapa ligero. Las primeras trece posiciones de su vector de entrada describen el movimiento y el espacio próximo al vehículo: velocidad longitudinal, velocidad angular y once distancias obtenidas mediante raycasting. Todas ellas se normalizan antes de ejecutar la inferencia para evitar que una magnitud domine a las demás únicamente por su escala.

Las ocho posiciones siguientes incorporan el contexto de señalización. Cuatro recogen si existe un semáforo activo, su estado numérico, la distancia a la línea de parada y la orden de parada forzada. Las otras cuatro forman una codificación *one-hot* que distingue entre semáforo, stop, ceda el paso o ausencia de control. La última posición es un sesgo constante. De este modo, la configuración vigente suma 22 entradas y permite que una misma lectura geométrica produzca respuestas distintas según la norma aplicable en ese punto.

La capa oculta contiene 16 neuronas y utiliza $\tanh$ como función de activación. La capa de salida contiene dos neuronas, también con $\tanh$, de modo que sus valores se encuentran en el rango $[-1, 1]$. La primera salida controla dirección y la segunda controla aceleración o frenado. La rutina de inferencia no fija internamente esas 22 entradas: deduce la dimensión esperada a partir de la longitud de la matriz de pesos de entrada y del número de neuronas ocultas. Esta guarda permite detectar de forma coherente modelos incompatibles si la configuración cambia.

3.5 ALGORITMOS EVOLUTIVOS

El entrenamiento se basa en población, selección y mutación. El gestor evolutivo crea una población de vehículos, les asigna pesos iniciales o mutados, ejecuta la simulación y registra el fitness final de cada individuo. Los mejores pesos se guardan en disco como modelos reutilizables, distinguiendo entre el mejor modelo histórico y el mejor de sesión.

La tasa de mutación documentada en la configuración actual se sitúa en torno al 4 % para mutaciones ordinarias, combinando variaciones pequeñas y mutaciones grandes excepcionales. Esta estrategia busca mantener diversidad sin destruir completamente comportamientos ya útiles.

3.6 ANÁLISIS DE DATOS

La instrumentación se diseñó para separar el detalle de cada vehículo de los resúmenes utilizados al comparar generaciones. Esta separación evita que los ficheros agregados crezcan innecesariamente y, al mismo tiempo, permite volver al registro individual cuando aparece una colisión, una penalización o un resultado anómalo. El proyecto genera tres ficheros principales:

- `Fitness_Debug.csv`, con métricas por coche: fitness, tiempo de vida, punto de aparición, razón de muerte, penalizaciones, bonus, datos de conducción, validación de stops y cedas, mutaciones y familia genealógica.
- `Training_Summary.csv`, con resumen por generación de entrenamiento.
- `Test_Summary.csv`, con resumen por generación de test, incluyendo aciertos, errores y tasas.

El análisis posterior se realiza mediante Python, NumPy, Matplotlib y Seaborn. El script `analyse_current.py` valida la calidad de los CSV, detecta automáticamente si la sesión es de entrenamiento o test, genera informes textuales y produce gráficas de evolución, muertes, distribución de fitness, ranking por spawn, control de entradas, convergencia, diversidad, correlaciones y diagnósticos específicos de paradas o cedas. El script `clean_unreal_log.py` permite limpiar logs de Unreal Engine conservando errores, avisos relevantes, mensajes de Blueprint y trazas propias del proyecto.

3.7 CONTROL DE VERSIONES

El trabajo se ha desarrollado con Git/GitLab en un contexto colaborativo. La documentación de seguimiento recoge tareas de limpieza de repositorio, configuración de `.gitignore`, eliminación de binarios pesados y organización de issues y milestones. Esta disciplina ha sido relevante porque el proyecto combina assets de Unreal, Blueprints, datos generados y modelos guardados.

Capítulo 4

Gestión del proyecto

4.1 METODOLOGÍA

El desarrollo se ha organizado mediante un enfoque iterativo e incremental. Las reuniones de seguimiento se celebraron de forma periódica, generalmente cada dos jueves, con participación de tutores, equipo técnico y personas vinculadas al proyecto original. En estas reuniones se revisaban avances, problemas, decisiones de arquitectura y siguientes objetivos. Esta dinámica se aproxima a una metodología ágil por sprints, aunque adaptada al contexto académico y a la disponibilidad del equipo.

La planificación inicial contemplaba una progresión amplia: semáforos y señales, IA vehicular, stops y cedas, perfiles de conductor, peatones, bicicletas, animales, diversidad de assets y optimización. El desarrollo real concentró el esfuerzo en construir una base sólida de IA vehicular, debido a la dificultad técnica de lograr navegación autónoma estable en la ciudad.

4.2 CRONOLOGÍA DE DECISIONES

La tabla 4.1 sintetiza los principales hitos recogidos en las actas de seguimiento.

Tabla 4.1: Principales hitos y decisiones documentadas en reuniones.

Fecha	Situación revisada	Decisión y consecuencia
27/11/2025	Integración de tráfico y elección de la primera IA.	Se prioriza la IA vehicular y se revisan semáforos dinámicos. Se descarta una malla dinámica para reconocer caminos por su coste de desarrollo.
11/12/2025	Representación de las rutas de la ciudad.	Se descarta mapear todo el escenario con splines y se prueba el <i>Path Finding</i> de Unreal. Se separan la decisión del modelo y la aplicación física del control.

Tabla 4.1: Principales hitos y decisiones documentadas en reuniones (continuación).

Fecha	Situación revisada	Decisión y consecuencia
17/12/2025	Interfaz entre la red y el vehículo.	Se fijan dos salidas continuas, dirección y aceleración/frenado, ambas dentro de $[-1, 1]$.
08/01/2026	Rigidez observada con <i>Path Finding</i> .	Se abandona esta opción como solución principal y se adopta entrenamiento evolutivo por prueba y error para aprender las maniobras básicas.
05/02/2026	Fallos en curvas e intersecciones.	Se introduce de forma provisional el etiquetado dinámico de bordes y se estudia cómo coordinar la elección de trayectoria con la percepción local.
19/02/2026	Falta de una validación conjunta de los subsistemas.	Se programa una demostración con red neuronal, sensores y bordes activos en todo el mapa.
05/03/2026	Demostración fallida y resultados difíciles de interpretar.	Se decide instrumentar la función de fitness y simplificar temporalmente los escenarios más complejos para localizar el origen de los fallos.
19/03/2026	Segunda demostración con navegación ya estable en buena parte del mapa.	Se acepta la base técnica, se planifica la reorganización del repositorio y se preparan la compilación para Windows y las siguientes mejoras.

Fuente: elaboración propia a partir de las actas de seguimiento.

4.3 ALCANCE

El alcance funcional completado se centra en conseguir que los vehículos de tráfico puedan aprender y circular dentro del escenario existente. El trabajo no modifica el vehículo conducido por el paciente ni rediseña la ciudad: actúa sobre los coches autónomos, su controlador y los elementos del mapa que les proporcionan información. El resultado es un controlador capaz de transformar las salidas de la red neuronal en dirección, aceleración y frenado sobre el sistema físico de Unreal Engine.

Para alimentar ese controlador se ha incorporado percepción local mediante ray-casting y se han normalizado las variables que entran en la red. El aprendizaje se realiza con poblaciones de vehículos, selección, mutación y elitismo. Los pesos obtenidos pueden guardarse y cargarse en sesiones posteriores, lo que evita reiniciar el proceso cada vez que se abre el simulador. La evaluación se apoya en una función de fitness que combina avance, centrado, velocidad, cumplimiento de señales y penalizaciones por comportamientos no deseados.

El alcance también comprende la interacción con semáforos, stops y cedas, incluida la detección de líneas de parada y la validación de las maniobras. Se han separado los puntos de aparición utilizados para entrenar de los reservados para test y se exportan métricas individuales y resúmenes a CSV. Esta instrumentación permite comparar sesiones y localizar intersecciones o puntos del mapa donde el modelo falla de forma repetida.

No forman parte de esta versión la implementación completa de peatones, bicicletas y animales, ni los perfiles de conductor diferenciados previstos en la propuesta inicial. Tampoco se aborda todavía una validación con pacientes. Estas ampliaciones requieren primero consolidar la navegación vehicular y disponer de un protocolo de prueba estable, por lo que se mantienen como trabajo posterior.

4.4 GESTIÓN DE RIESGOS

La tabla 4.2 relaciona los riesgos más relevantes con su impacto, probabilidad y estrategia de mitigación.

Tabla 4.2: Riesgos principales del proyecto.

Riesgo	Impacto	Prob.	Mitigación
Rigidez de navegación determinista	Comportamiento poco realista y baja utilidad clínica	Alta	Pivotar a percepción local con sensores y red neuronal.
Fitness mal calibrado	Aprendizaje de estrategias no deseadas	Alta	Instrumentar componentes, exportar CSV y analizar penalizaciones por familia de muerte.
Coste computacional	Entrenamientos lentos y dependencia de GPU/CPU	Media	Poblaciones moderadas, análisis offline y propuesta futura de modo sin renderizado.

Tabla 4.2: Riesgos principales del proyecto (continuación).

Riesgo	Impacto	Prob.	Mitigación
Complejidad de intersecciones	Colisiones y violaciones normativas en cruces	Alta	Bordes dinámicos, zonas de cruce, validación de stops/cedas y separación train/test.
Sobreajuste a spawns concretos	Buen rendimiento local pero mala generalización	Media	Normalización por punto de aparición y test con spawns diferenciados.
Pérdida de conocimiento entrenado	Reinicio de aprendizaje entre sesiones	Media	Persistencia de modelos SuperCarModel y SessionCarModel.

Fuente: elaboración propia a partir del seguimiento técnico del proyecto.

4.5 ESTIMACIÓN DE RECURSOS Y PRESUPUESTO

El proyecto se ha desarrollado durante las prácticas y el TFG, sin contratación externa ni compra específica de material. El recurso de mayor peso ha sido el tiempo dedicado a implementar Blueprints, ejecutar entrenamientos, revisar los resultados y documentar cada cambio. Una parte relevante de ese tiempo no corresponde a programación directa: los entrenamientos ocupan el equipo durante periodos prolongados y obligan a repetir pruebas cuando cambia la función de fitness o la geometría de una intersección.

El trabajo técnico se ha realizado con un equipo capaz de ejecutar Unreal Engine 5 y las físicas de varios vehículos de forma simultánea. Para el control de versiones se ha utilizado GitLab y el análisis se ha realizado con Python y bibliotecas de uso libre. Unreal Engine tampoco ha generado un coste de licencia en el contexto académico del proyecto. Por tanto, no existen costes directos de software ni de infraestructura en esta fase; el hardware ya estaba disponible y solo correspondería imputar su amortización y consumo eléctrico si se elaborase un presupuesto comercial.

En la memoria final, la valoración económica deberá convertir las horas efectivas de desarrollo y experimentación a un coste de ingeniería acordado con el tutor, y añadir la parte proporcional del equipo utilizado. Mantener separadas ambas partidas evita presentar como gasto real una licencia que no se ha pagado y permite actualizar el presupuesto cuando se cierre el registro definitivo de dedicación.

Capítulo 5

Núcleo del trabajo

5.1 DESCRIPCIÓN GENERAL DE LA SOLUCIÓN

La solución actual se estructura como un sistema de agentes autónomos entrenados dentro del simulador. Cada vehículo dispone de un controlador de IA que percibe el entorno, ejecuta una red neuronal, aplica acciones al sistema físico y registra métricas de comportamiento. Por encima de los vehículos existe un gestor evolutivo que crea poblaciones, asigna pesos, evalúa resultados, selecciona los mejores individuos y guarda modelos.

El sistema se completa con elementos de tráfico que comunican contexto a los controladores: semáforos, señales de stop, señales de ceda y zonas de cruce. Estos elementos no son meros objetos visuales, sino actores con lógica propia que activan zonas de detección, informan al vehículo de líneas de parada, controlan estados y aplican penalizaciones o bonus según el comportamiento observado.

5.2 SUBSISTEMAS

5.2.1 Gestor evolutivo

`BP_EvolutionManager` coordina el ciclo de entrenamiento y las ejecuciones de test. Al comenzar comprueba si existe un modelo histórico en `SuperCarModel`. Si lo encuentra, carga sus pesos; en caso contrario, inicializa una red aleatoria que servirá como punto de partida. Después actualiza la lista de puntos de aparición permitidos para el modo activo y crea la población de vehículos, asignando a cada coche su controlador y su red.

Durante el entrenamiento conviven individuos procedentes del mejor modelo histórico, del mejor modelo obtenido en la sesión y de inicializaciones aleatorias. Los primeros se conservan o se mutan para explotar soluciones que ya han funcionado, mientras que los aleatorios mantienen capacidad de exploración. La generación termina cuando todos los vehículos han finalizado o han sido eliminados por alguna condición de muerte. En ese momento el gestor ordena los resultados, actualiza los modelos persistidos cuando corresponde, destruye la población anterior e inicia la siguiente generación.

El modo de test reutiliza la misma infraestructura, pero desactiva la mutación y carga un modelo fijo. También emplea un conjunto de puntos de aparición distinto del utilizado para entrenar. Esta separación evita que una mejora aparente proceda únicamente de evaluar el coche en los recorridos que ya han condicionado su aprendizaje. Al terminar cada bloque, el gestor exporta tanto el resumen de la ejecución como los registros individuales necesarios para el análisis.

Una decisión relevante es la normalización por punto de aparición. La ciudad contiene zonas de dificultad desigual y comparar directamente el fitness de dos spawns puede favorecer al que comienza en una ruta sencilla. Por ello se conserva el mejor valor observado en cada punto y el resultado de un coche se relaciona con esa referencia local antes de seleccionar sus pesos. La normalización no elimina las diferencias del mapa, pero reduce su influencia en la elección del individuo que pasa a la siguiente generación.

5.2.2 Controlador de IA y red neuronal

BP_AiController actúa como intermediario entre red neuronal y vehículo físico. En cada tick comprueba que el vehículo controlado existe y no está muerto, prepara las entradas de red, ejecuta la inferencia y aplica las salidas al componente de movimiento.

Para evitar saltos bruscos, el controlador dispone de variables de suavizado como MaxSteeringRate y MaxThrottleRate. La versión documentada mantiene un seguimiento estable entre objetivo y entrada aplicada, con gap medio cero en los informes finales, lo que indica que la señal de control se está transmitiendo sin divergencias instrumentales.

La inferencia se calcula manualmente sobre arrays de pesos:

$$h_j = \tanh \left(\sum_i w_{j,i}^{IH} x_i \right) \quad (5.1)$$

$$y_k = \tanh \left(\sum_j w_{k,j}^{HO} h_j \right) \quad (5.2)$$

La ecuación 5.1 calcula las activaciones ocultas y la ecuación 5.2 obtiene las dos salidas de control, donde x_i son las entradas normalizadas, h_j las neuronas ocultas e y_k las salidas. Los superíndices identifican los dos bloques de pesos de la red. *IH* significa *Input-Hidden* y corresponde a las conexiones entre la entrada y la capa oculta. *HO*, denominado *WeightsH0* en los Blueprints y anotado en ocasiones como *OH*, corresponde a las conexiones *Hidden-Output* entre la capa oculta y las dos salidas.

Los pesos se almacenan en arrays unidimensionales. Por esa razón, el número de entradas esperado se obtiene dividiendo la longitud de `WeightsIH` entre el número de neuronas ocultas. A continuación se comprueba que ese resultado coincide con el tamaño del vector construido por el controlador, que `WeightsIH` contiene exactamente $n_{entradas} \times n_{ocultas}$ valores y que `WeightsHO` contiene $n_{ocultas} \times n_{salidas}$. Este cálculo sustituye a una comparación rígida con 22 y permite detectar un modelo antiguo aunque cambie la arquitectura. Si faltan pesos o las dimensiones no son coherentes, se activa una guarda; después de cinco errores consecutivos, el coche se elimina con la razón de muerte `INVALID_BRAIN` para evitar que continúe circulando con una red inválida.

5.2.3 Percepción mediante sensores

El controlador lanza once trazas en abanico sobre 180 grados. Cada sensor devuelve una distancia normalizada hasta el primer elemento relevante. Los bordes de intersección se tratan con lógica específica: un borde virtual solo interviene en la lectura cuando su intención de dirección y su trigger coinciden con la decisión vigente del vehículo. Los bordes asociados a otras trayectorias se ignoran, de modo que una misma intersección puede contener límites sensoriales distintos para cada maniobra.

La decisión se modela mediante la enumeración `E_DirectionIntent`, con hasta cuatro alternativas de salida. Al entrar en un `BP_IntersectionTrigger`, el vehículo selecciona aleatoriamente una de las opciones habilitadas para el modo de entrenamiento o de test. El controlador conserva tanto la dirección elegida como el trigger que la originó. Cada `BP_IntersectionBorders` declara la pareja dirección–trigger a la que pertenece, lo que permite que los rayos perciban únicamente los límites que delimitan la trayectoria seleccionada.

Las entradas sensoriales se combinan con información de tráfico. Si el vehículo entra en la zona de un semáforo, stop o ceda, el actor correspondiente comunica al controlador el tipo de control, la línea de parada y la distancia de entrada. Esta mezcla de percepción geométrica y contexto normativo permite que la red no dependa únicamente de distancia a obstáculos.

5.2.4 Sistema de fitness

La función de fitness se ha convertido en uno de los elementos centrales del proyecto. Las primeras versiones premiaban principalmente el avance, pero ese criterio por sí solo permitía comportamientos poco útiles: circular demasiado rápido cerca de un obstáculo, corregir hacia el lado equivocado o sobrevivir sin progresar. La función actual combina el avance con la velocidad útil y el centrado estimado a partir de los sensores laterales. También valora que la velocidad sea coherente con la distancia frontal disponible, de modo que aproximarse a un obstáculo sin reducir la marcha deje de ser ventajoso.

Las penalizaciones se aplican cuando el vehículo marcha hacia atrás, permanece atascado, abandona la calzada o colisiona. Hay comprobaciones específicas para las correcciones de dirección y para el incumplimiento de semáforos, stops, cedas y zonas no navegables. Completar correctamente un stop o un ceda produce una bonificación, pero esa recompensa se registra por separado para poder comprobar que no compensa una infracción grave ocurrida en otro momento de la ejecución.

Además del valor final, cada coche acumula métricas de aceleración, frenado, espera, dirección y contexto de parada. Esta instrumentación permite reconstruir qué componentes han dominado el resultado y distinguir, por ejemplo, una finalización por tiempo con recorrido útil de un vehículo inmóvil ante una señal. El desglose fue decisivo después de la demostración fallida del 5 de marzo de 2026, cuando la observación visual no bastó para localizar si el problema procedía de los sensores, de las restricciones o del propio cálculo de fitness.

5.2.5 Semáforos

El sistema de semáforos se compone de un controlador de intersección y de actores de semáforo. El controlador alterna fases norte-sur y este-oeste, incluyendo verde, ámbar y todo rojo. Cada semáforo mantiene materiales dinámicos para representar estado visual y una zona de sensor que detecta vehículos.

Cuando un vehículo entra en la zona, el semáforo comunica estado, línea de parada y distancia. Si el semáforo está en rojo o ámbar restrictivo, el controlador activa contexto de parada. Al salir de la zona, se comprueba si el vehículo atravesó la línea a una velocidad legal. El sistema aprende umbrales de velocidad legal a partir de muestras previas, evitando depender siempre de un valor fijo.

5.2.6 Stop y ceda el paso

Las señales BP_Sign_Stop y BP_Sign_Yield amplían la interacción normativa. En stop se exige detención suficiente dentro de una ventana dinámica antes de la línea, espera mínima y comprobación de cruce libre. Si se cumple, se aplica bonus y se libera el vehículo; si no, se aplica penalización y muerte por STOP_VIOLATION. En ceda se monitoriza velocidad, aproximación y disponibilidad de la zona de cruce, con penalización por YIELD_VIOLATION y bonus al completar correctamente.

La diferencia entre stop y ceda es metodológicamente importante. El stop requiere detención explícita; el ceda permite continuar si la velocidad y la prioridad son compatibles con la seguridad del cruce.

5.3 PIPELINE EXPERIMENTAL

Cada ejecución genera un registro por vehículo y un resumen por generación. Una vez terminada la sesión, los scripts de Python validan que ambos CSV contienen el número de filas esperado y que sus identificadores pueden alinearse. Solo después de esa comprobación se calculan los indicadores y se generan las gráficas. De esta manera, una fila incompleta o una generación duplicada no queda oculta dentro de una media agregada.

El diagnóstico comienza con *BestFit*, *MeanFit* y tiempo de vida, pero no termina en esas tres cifras. Las razones de muerte se agrupan por familias para separar colisiones, finalizaciones por tiempo y fallos normativos; los bonus de stop y ceda se contrastan con el contexto de frenado; y los resultados se desglosan por punto de aparición. También se revisan las correlaciones entre los componentes del fitness y la estabilidad con la que las consignas de dirección y aceleración llegan al vehículo.

Este procedimiento ha permitido utilizar los datos para decidir el siguiente cambio de implementación. Cuando un indicador empeora, se vuelve al CSV individual para localizar actores, intersecciones o spawns concretos; después se modifica el Blueprint correspondiente y se repite el entrenamiento o el bloque de test. El pipeline cierra así el ciclo entre implementación, simulación y análisis sin depender únicamente de lo observado en pantalla.

Capítulo 6

Resultados

6.1 DATOS ANALIZADOS

La evaluación de referencia utiliza la sesión `train_2026-06-08_12-16`, con 2067 generaciones y 66144 registros de vehículos. La hora incluida en el nombre sirve únicamente para identificar la carpeta y ordenar las ejecuciones; no se considera una variable experimental. Para test no se selecciona una única sesión favorable, sino que se agregan las seis realizadas el 8 de junio con el mismo modelo y la misma configuración. En conjunto contienen 30 ejecuciones, 1560 evaluaciones y 30 observaciones de cada uno de los 52 puntos de aparición.

Todos los informes automáticos del bloque principal califican la calidad de los datos como correcta y no muestran alertas fuertes de desalineación entre `Summary` y `Debug`. Las sesiones de test cargan modelos persistidos, tienen la mutación desactivada y mantienen el mismo tamaño de población, por lo que se analizan como repeticiones bajo una configuración común. Para contextualizar su evolución, también se agregan los ocho tests disponibles del 2 de junio y los ocho del 5 de junio.

6.2 RESUMEN CUANTITATIVO

La tabla 6.1 confirma la ausencia de fitness medio negativo y de muertes LAZY, así como una incidencia legal baja. No obstante, las colisiones constituyen aproximadamente siete de cada diez finalizaciones tanto durante el aprendizaje como en test. El mayor *MeanFit* de test respecto al entrenamiento no implica que el modelo mejore durante la evaluación: responde a que ambos modos utilizan poblaciones, condiciones de ejecución y objetivos de medición diferentes.

6.3 EVOLUCIÓN DEL ENTRENAMIENTO

El mejor fitness de la sesión se alcanza en la generación 117, con 112447.60, mientras que el fitness medio máximo aparece en la generación 36, con 29470.20. La media de *MeanFit* es 16162.51 y ninguna generación presenta un valor medio negativo.

Tabla 6.1: Resultados principales del entrenamiento y del bloque de test del 8 de junio de 2026.

Métrica	Entrenamiento	Test agregado
Generaciones o ejecuciones	2067	30
Registros de vehículos	66144	1560
<i>BestFit</i> máximo	112447.60	85530.55
<i>MeanFit</i> medio	16162.51	31222.36
<i>MeanFit</i> mediano	16007.06	31176.54
Tiempo medio de vida	45.10 s	53.13 s
Ejecuciones con <i>MeanFit</i> negativo	0.00 %	0.00 %
Colisiones	69.89 %	74.62 %
Fallos legales	0.29 %	0.06 %
LAZY	0.00 %	0.00 %
Tasa de acierto	—	25.19 %

Fuente: elaboración propia a partir de los CSV del proyecto.

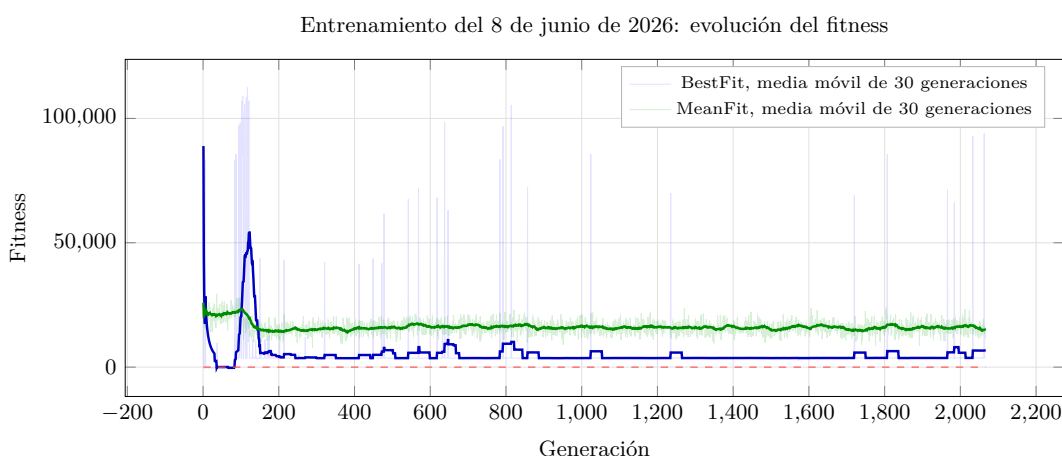


Figura 6.1: Tendencia del fitness en el entrenamiento del 8 de junio de 2026. Fuente: análisis generado a partir de los CSV del proyecto.

El análisis conjunto de la figura 6.1 y de la serie temporal de los CSV obliga a matizar esos máximos. Entre las primeras y las últimas 50 generaciones, el fitness medio desciende un 27.5 %, de 21239.35 a 15406.23, mientras el tiempo medio de vida aumenta un 26.8 %, de 36.35 a 46.10 segundos. El informe detecta una tendencia reciente positiva, pero no convergencia clara. Por tanto, el entrenamiento prolonga la supervivencia y mantiene estabilidad frente a penalizaciones extremas, aunque no produce una mejora monótona de la función de aptitud.

La causa LAZY desaparece por completo en los 66144 registros. Los fallos legales bajan al 0.29 %, frente al 6.82 % de la sesión de referencia del 2 de junio. Sin embargo, el analizador identifica 8691 posibles atascos ante stop entre 19697 finalizaciones por tiempo, un 44.12 %. Se trata de una heurística y no de 8691 infracciones confirmadas, pero su magnitud aconseja revisar los umbrales de detección y realizar tests dirigidos sobre esos puntos.

6.4 EVALUACIÓN DE TEST

En las seis sesiones del 8 de junio se registran 393 aciertos de 1560 evaluaciones, equivalentes al 25.19 %. El intervalo de confianza de Wilson al 95 % se sitúa entre el 23.10 % y el 27.41 %. Las tasas por sesión oscilan entre el 20.38 % y el 30.00 %, con una mediana del 25.58 %; esta dispersión confirma que una sola sesión de 260 vehículos puede sobreestimar o infraestimar el rendimiento.

Para comparar fechas se recalculan todos los indicadores directamente desde `Fitness_Debug.csv`. Este criterio evita depender de los totales de `Summary`: el bloque del 2 de junio contiene una fila debug menos de la declarada y una sesión del 5 de junio conserva una generación duplicada en su resumen. Estas incidencias no aparecen en el bloque del 8 de junio. El fitness final medio de la tabla 6.2 se obtiene promediando `Fitness_Final` por vehículo; es, por tanto, un indicador distinto del *MeanFit* calculado en los resúmenes por generación.

Tabla 6.2: Evolución de los bloques de test disponibles en junio de 2026.

Fecha	Evaluaciones	Acierto	Fitness final medio	Vida media
2 de junio	1975	16.51 %	27195.46	46.16 s
5 de junio	2080	18.51 %	26599.02	44.98 s
8 de junio	1560	25.19 %	31295.04	53.13 s

Fuente: elaboración propia a partir de `Fitness_Debug.csv`.

Respecto al bloque del 5 de junio, el resultado más reciente mejora 6.68 puntos porcentuales en acierto, un 17.7 % en fitness medio y un 18.1 % en tiempo de vida. Frente al bloque del 2 de junio, la mejora de acierto es de 8.69 puntos. Esta comparación es más robusta que la tasa del 30 % anteriormente documentada, que procedía de una sola sesión de 260 evaluaciones.

6.5 RAZONES DE FINALIZACIÓN

De las 1560 evaluaciones, 1164 terminan en colisión, el 74.62 %, y 393 alcanzan el límite de tiempo, el 25.19 %. También se observan dos vuelcos, que suponen el 0.13 %, y una violación normativa, el 0.06 %. El foco de colisión más frecuente es `NavModifierVolume19`, con 163 casos y un 10.45 % del total de evaluaciones. Le siguen `NavModifierVolume53`, con 124 casos y un 7.95 %, y `BP_IntersectionBorder203`, con 117 casos y un 7.50 %.

La concentración en actores y bordes concretos indica que la principal limitación ya no es un fallo general de movimiento, sino la robustez geométrica en determinadas aproximaciones e intersecciones. Esta evidencia permite priorizar la inspección de esos elementos del mapa, los sensores locales y la cobertura de sus spawns asociados.

6.6 COMPORTAMIENTO ANTE SEÑALIZACIÓN

En las seis sesiones, el uso de freno dentro de contextos de parada se mantiene entre el 99.33 % y el 99.50 %, mientras el uso global de freno se sitúa entre el 17.55 % y el 19.31 %. Además, el bloque completo solo registra una finalización clasificada como violación normativa. Estos datos respaldan que el contexto de semáforos, stops y cedas modifica de forma efectiva la acción del controlador.

Los informes detectan cinco candidatos fuertes de atasco en stop y dos en ceda entre las 393 finalizaciones por tiempo. Son pocos en test, pero no deben equipararse automáticamente con una conducción correcta: alcanzar el límite temporal puede representar tanto supervivencia útil como inmovilidad prolongada. La siguiente validación deberá separar explícitamente ambos resultados mediante distancia recorrida, progresión de ruta y tiempo detenido.

6.7 DIFICULTAD POR PUNTOS DE APARICIÓN

El diseño del bloque de test asigna 30 observaciones a cada punto de aparición. Mantener el mismo número evita que un spawn utilizado con mayor frecuencia tenga más peso en el resultado agregado. En los CSV, los identificadores siguen el patrón `BP_SpawnPointN`; el número permite volver desde el informe al actor concreto del mapa y revisar su orientación, la carretera inicial y la primera intersección que encuentra el vehículo.

Los puntos 9, 26 y 25 obtienen los mayores fitness medios, con 61384.0, 60802.3 y 60615.0 respectivamente. En el extremo inferior se encuentran los puntos 5, 39, 23 y 2, cuyos valores son 2048.0, 5509.5, 5732.5 y 5920.7. La separación entre el mejor y el peor supera los 59000 puntos, una diferencia demasiado grande para interpretar la tasa global sin estudiar también la localización de inicio.

Este resultado no permite concluir por sí solo que la red haya aprendido unas zonas y no otras. Parte de la diferencia puede proceder de la geometría inmediata, de un borde de intersección mal configurado o de que el tiempo disponible favorezca recorridos más largos. Por ello, los puntos de menor rendimiento se utilizarán para pruebas dirigidas en las que se revisen colisiones, distancia recorrida y primera maniobra. Los puntos 9, 26 y 25 servirán como control para comprobar que una corrección local no introduce una regresión en recorridos que ahora funcionan mejor.

6.8 CONCLUSIÓN EXPERIMENTAL

La evidencia nueva refuerza la validez funcional del sistema: el test agregado mejora los tres indicadores principales respecto a los bloques del 2 y 5 de junio, elimina LAZY y mantiene una incidencia normativa muy baja. Al mismo tiempo, evita presentar como definitiva una ejecución aislada favorable. La tasa de acierto del 25.19% y el predominio de las colisiones muestran que el controlador aún no es suficientemente robusto para su uso como tráfico autónomo final en sesiones clínicas. Las siguientes iteraciones deben centrarse en intersecciones concretas, spawns de bajo rendimiento, discriminación entre supervivencia y atasco, y validaciones repetidas con el mismo protocolo.

Capítulo 7

Normativa y accesibilidad

7.1 PROTECCIÓN DE DATOS

El proyecto se orienta a una aplicación potencial en rehabilitación cognitiva, por lo que debe contemplar desde el diseño la protección de datos personales y, especialmente, de datos de salud. Durante el desarrollo descrito en esta memoria no se ha trabajado con datos reales de pacientes. Los registros analizados corresponden a vehículos autónomos simulados, métricas de entrenamiento y resultados técnicos.

En una futura validación clínica, cualquier uso con personas usuarias deberá ajustarse al Reglamento General de Protección de Datos y a la Ley Orgánica 3/2018 de Protección de Datos Personales y garantía de los derechos digitales [5, 6]. Será necesario definir responsable del tratamiento, finalidad, base jurídica, minimización de datos, conservación, consentimiento informado cuando proceda, medidas de seudonimización y control de acceso. También deberá diferenciarse entre datos necesarios para la evaluación terapéutica y métricas técnicas internas del simulador.

7.2 SEGURIDAD Y USO RESPONSABLE

El simulador no sustituye por sí mismo una evaluación clínica ni una autorización administrativa para conducir. Su función debe entenderse como apoyo a profesionales, entrenamiento controlado y herramienta de observación. Cualquier conclusión sobre capacidad real de conducción debería depender de especialistas y procedimientos establecidos.

Desde el punto de vista técnico, el sistema autónomo genera tráfico artificial. Por tanto, sus fallos no suponen riesgo físico directo durante el entrenamiento virtual, pero sí pueden afectar a la validez de la experiencia. Es importante que las sesiones con pacientes utilicen modelos suficientemente probados para evitar comportamientos incoherentes que confundan al usuario o introduzcan estímulos no deseados.

7.3 ACCESIBILIDAD

La accesibilidad del simulador debe analizarse en relación con las personas a las que se dirige, que pueden presentar alteraciones de atención, memoria, percepción o movilidad. En este TFG no se ha rediseñado la interfaz utilizada por el paciente, por lo que no puede afirmarse que el producto completo haya sido validado como accesible. La aportación realizada se concentra en el tráfico autónomo y afecta de forma indirecta a la carga cognitiva de cada sesión.

El número de vehículos, los puntos del mapa y la frecuencia de situaciones normativas deben poder ajustarse para que la dificultad aumente de forma progresiva. También es importante repetir un mismo escenario bajo condiciones comparables: si el tráfico cambia sin control, resulta difícil saber si una variación en el resultado procede del paciente o de una sesión más exigente. La separación entre entrenamiento y test de la IA y el registro de eventos sientan una base técnica para construir esas repeticiones, aunque todavía falta trasladar estos parámetros a una configuración manejable por el profesional.

La interacción debe contemplar periféricos adaptados, instrucciones breves y legibles y una presentación que no añada estímulos irrelevantes. El rendimiento gráfico y la duración de la prueba también deben vigilarse para reducir fatiga o mareo. Por último, los datos exportados no deberían mostrarse al especialista como un volcado técnico de CSV, sino como indicadores comprensibles y vinculados a maniobras concretas. Estos requisitos deberán revisarse con profesionales y usuarios de ASPAYM antes de utilizar el simulador en una validación clínica.

Capítulo 8

Conclusiones y líneas futuras

8.1 CONCLUSIONES

El punto de partida del trabajo era una ciudad preparada para la conducción del usuario, pero sin un sistema de tráfico capaz de recorrerla de forma autónoma. Las primeras pruebas confirmaron que trasladar soluciones habituales de navegación de personajes a un coche con físicas producía recorridos rígidos y problemas en los cruces. La decisión de sustituir las rutas cerradas por percepción local y una política neuronal permitió avanzar, aunque el resultado no debe interpretarse como un controlador ya resuelto.

La arquitectura implementada conecta once sensores, variables dinámicas y contexto de señalización con una red de 22 entradas, 16 neuronas ocultas y dos salidas. Las consignas obtenidas se aplican al componente físico del vehículo, mientras el gestor evolutivo se ocupa de crear poblaciones, seleccionar modelos y conservar pesos entre sesiones. Esta integración funciona dentro del escenario real del simulador y no en una prueba aislada, que era una de las dificultades principales del proyecto.

El desarrollo también ha puesto de manifiesto que la función de fitness y el registro de datos forman parte de la solución, no son tareas posteriores. Una demostración visual podía parecer correcta y ocultar estancamientos, penalizaciones mal equilibradas o resultados concentrados en unos pocos puntos del mapa. La exportación por vehículo permitió revisar esas situaciones y motivó cambios concretos en el fitness, en los bordes de intersección y en la separación entre entrenamiento y test.

Los resultados del 8 de junio muestran una mejora respecto a los bloques del 2 y del 5 de junio. Las seis sesiones de test reúnen 1560 evaluaciones y alcanzan un 25.19 % de acierto, con un intervalo de confianza del 95 % entre el 23.10 % y el 27.41 %. En entrenamiento desaparece la causa de muerte LAZY y disminuyen los fallos legales. Sin embargo, el 74.62 % de las evaluaciones de test termina en colisión y el rendimiento cambia de forma notable entre puntos de aparición. Además, la evolución del fitness no presenta una convergencia clara.

Por tanto, se ha obtenido una base funcional para generar, entrenar y medir tráfico autónomo, pero todavía no un sistema suficientemente robusto para una sesión clínica. La utilidad inmediata del trabajo reside en que los fallos ya pueden localizarse y compararse con un protocolo común. Las siguientes iteraciones deberán demostrar que las mejoras se mantienen en intersecciones y puntos de inicio distintos antes de aumentar la densidad de tráfico o incorporar nuevos tipos de agentes.

8.2 APORTACIONES REALIZADAS

La primera aportación es el controlador neuronal integrado en el vehículo de Unreal Engine. El trabajo incluye la construcción del vector de 22 entradas, la inferencia de la red y la aplicación suavizada de las dos salidas al sistema Chaos Vehicle. Los sensores no se limitan a detectar obstáculos: en las intersecciones filtran los bordes según la maniobra elegida y combinan esa geometría con el estado de semáforos, stops y cedas.

La segunda aportación es el proceso de aprendizaje dentro del propio simulador. El gestor crea poblaciones, conserva individuos, aplica mutaciones y guarda por separado el mejor modelo histórico y el mejor de la sesión. La normalización por punto de aparición intenta reducir la ventaja de comenzar en una zona sencilla, y el uso de spawns distintos en test permite evaluar el modelo fuera del conjunto que condicionó su entrenamiento.

La tercera aportación es la infraestructura de evaluación. Los Blueprints exportan el resultado de cada vehículo y los resúmenes de generación; posteriormente, los scripts comprueban la calidad de los CSV, agrupan razones de muerte y producen comparaciones por sesión y por spawn. Esta conexión entre simulación y análisis permite justificar una modificación con datos y comprobar si corrige el problema sin introducir una regresión en otras zonas.

Finalmente, las actas y descripciones fechadas conservan la evolución técnica del proyecto. Gracias a ellas puede reconstruirse por qué se abandonó *Path Finding*, cuándo se reformuló el fitness o cómo se introdujeron los bordes dependientes de la trayectoria. Esta trazabilidad resulta especialmente útil en un desarrollo compartido que deberá continuar después del TFG.

8.3 PROBLEMAS ENCONTRADOS

El primer problema fue adaptar la navegación de Unreal Engine a un vehículo con inercia y radio de giro. NavMesh y *Path Finding* encontraban un destino, pero no producían una conducción natural. Abandonar esa vía obligó a trabajar conjuntamente con Chaos Vehicle, el control mediante Blueprints y los sensores de calzada.

Las intersecciones concentraron buena parte de los fallos posteriores. Un mismo cruce debía ofrecer límites distintos según la salida elegida, y esos límites tenían que coincidir con el trigger que había originado la decisión. Un borde mal asociado podía desviar los rayos o provocar una colisión repetida. La introducción de intenciones de dirección resolvió parte del problema, pero los resultados por spawn muestran que la geometría todavía necesita revisión local.

El ajuste del fitness fue otra dificultad importante. Premiar avance y supervivencia generó estrategias no previstas, entre ellas permanecer detenido o compensar una colisión con recompensas acumuladas. Para distinguir estos casos fue necesario exportar penalizaciones, contexto de parada y razones de muerte. Esta instrumentación aumentó el tamaño y la complejidad de los datos, pero evitó continuar ajustando el modelo únicamente a partir de la impresión visual.

Por último, cada entrenamiento exige ejecutar física y renderizado para una población completa. Este coste limita la velocidad con la que pueden probarse cambios y hace especialmente caro descubrir tarde un error de configuración. La persistencia de modelos, la validación de los CSV y los bloques de test repetidos reducen ese riesgo, aunque sigue pendiente un modo de entrenamiento más rápido.

8.4 LÍNEAS FUTURAS

La siguiente iteración debería comenzar por los puntos 5, 39, 23 y 2 y por los actores que concentran más colisiones. En cada caso conviene separar un posible error de geometría de un fallo del controlador: revisar orientación y bordes del spawn, repetir la primera maniobra con el mismo modelo y comparar distancia recorrida, tiempo detenido y causa de finalización. Solo después tendría sentido incorporar esos puntos a un entrenamiento dirigido.

En paralelo debe mejorarse la evaluación de paradas. Alcanzar el límite de tiempo puede significar que el coche ha sobrevivido o que ha quedado inmóvil ante una señal. Registrar progreso de ruta y duración continua del atasco permitiría separar ambos casos y ajustar el fitness sin penalizar una espera correcta. Los bloques de test deberán conservar el mismo tamaño, modelo y conjunto de spawns para que los intervalos de confianza sigan siendo comparables.

Para reducir el tiempo entre iteraciones se plantea un modo sin renderizado o con la simulación desacoplada de la presentación. Esta mejora permitiría entrenar más generaciones y reservar la ejecución completa para validar física, señalización y comportamiento visual. También sería conveniente automatizar pruebas sobre intersecciones concretas antes de lanzar una sesión larga.

Los perfiles de conductor normal, inseguro y agresivo, la interacción entre vehículos y la incorporación de peatones o bicicletas deben abordarse cuando la base vehicular sea estable. Añadir agentes antes de resolver las colisiones actuales dificultaría atribuir cada fallo a su causa. Finalmente, cualquier uso con pacientes requerirá definir con ASPAYM un protocolo de sesión, criterios de accesibilidad e indicadores comprensibles para los profesionales.

Agradecimientos

Pendiente de redacción definitiva.

Bibliografía

- [1] L. Fuente, *CognitiveCarSimulatorReloaded - IA*, propuesta de proyecto, 2025.
- [2] E. Romero Yuste, *CognitiveCarSimulator*, Trabajo de Fin de Grado, Grado en Desarrollo de Aplicaciones Interactivas 3D y Videojuegos, Escuela Politécnica Superior de Zamora, 2025.
- [3] Epic Games, *Unreal Engine 5 Documentation*, Epic Developer Community. Disponible en: documentación oficial de Unreal Engine (consulta: 8 de junio de 2026).
- [4] Epic Games, *Chaos Vehicles*, Epic Developer Community. Disponible en: documentación oficial de Chaos Vehicles (consulta: 8 de junio de 2026).
- [5] Parlamento Europeo y Consejo de la Unión Europea, *Reglamento (UE) 2016/679, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos*, Diario Oficial de la Unión Europea, 2016. Disponible en: EUR-Lex (consulta: 8 de junio de 2026).
- [6] Jefatura del Estado, *Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales*, Boletín Oficial del Estado, núm. 294, 6 de diciembre de 2018. Disponible en: BOE (consulta: 8 de junio de 2026).

Apéndice A

Fuentes analizadas

A.1 DOCUMENTACIÓN ADMINISTRATIVA Y DE FORMATO

Para la redacción de esta memoria se han revisado los documentos incluidos en INFO/UTILES. Entre ellos destacan:

- portada y normas básicas de estilo de la Escuela de Ingenierías Industrial, Informática y Aeroespacial;
- solicitud de preinscripción del TFG, con título, descripción y tutoría;
- resolución favorable de preinscripción de fecha 18 de mayo de 2026;
- propuesta *CognitiveCarSimulatorReloaded - IA*;
- memoria previa *CognitiveCarSimulator*;
- informe de seguimiento de prácticas HP SCDS;
- memoria final de prácticas del alumno;
- estructura orientativa propuesta por el tutor.

A.2 ACTAS DE REUNIÓN

Se han analizado ocho actas de seguimiento, comprendidas entre el 27/11/2025 y el 19/03/2026. Estas actas permiten justificar las decisiones de arquitectura y los cambios de enfoque realizados durante el proyecto.

A.3 DESCRIPCIONES DEL PROGRAMA

Las descripciones fechadas de los Blueprints se utilizaron para reconstruir el estado de la implementación sin depender únicamente de la versión abierta en el editor. Para el gestor se revisó la carpeta `EVOLUTIONMANAGER`, donde se documentan la creación de generaciones, la asignación de modelos, la mutación, la persistencia y el cambio al modo de test. La carpeta `REDNEURONAL` permitió comprobar la composición de las entradas, las dimensiones de `WeightsIH` y `WeightsHO`, la inferencia y las guardas frente a modelos incompatibles.

El cálculo de aptitud y su instrumentación proceden de `FITNESS`. Estas descripciones detallan las recompensas, las penalizaciones y los campos exportados a CSV. Para la señalización se consultó `INTERSECTIONS`, incluidas las actualizaciones del 5 de junio de 2026 sobre semáforos, stops, cedas, zonas de cruce y validación normativa. Por último, `OTROS` reúne macros auxiliares, trazas, detección de estancamiento y tareas pendientes que ayudan a interpretar dependencias entre módulos.

A.4 RESULTADOS EXPERIMENTALES

Los resultados documentados proceden de `INFO/ANALISIS_DATOS/Analisis`. La referencia de entrenamiento es `train_2026-06-08_12-16`, que contiene 2067 generaciones y 66144 registros de vehículos. La marca horaria forma parte del identificador de la carpeta y no se utiliza como variable en el análisis. La sesión se eligió porque es la asociada al modelo evaluado y dispone de los CSV e informes necesarios para reproducir los indicadores del capítulo 6.

La evaluación principal reúne las seis carpetas de test del 8 de junio. En total contienen 30 ejecuciones y 1560 registros, con la misma cantidad de observaciones para cada uno de los 52 puntos de aparición. No se eligió la sesión con mejor tasa, sino que se agregaron las seis para reducir el efecto de una ejecución aislada. Como referencia histórica se utilizaron también los ocho tests del 2 de junio y los ocho del 5 de junio, con 1975 y 2080 registros de detalle respectivamente.

Los entrenamientos `train_2026-06-02_12-00` y `train_2026-06-05_11-47` se consultaron para contextualizar la evolución anterior. Los informes automáticos califican como correctos los datos del bloque principal y no detectan desalineaciones fuertes entre `Summary` y `Debug`. La figura del capítulo de resultados se ha regenerado en formato PDF vectorial a partir del `Training_Summary.csv` archivado de la sesión del 8 de junio.